

Meta Multi-Asset Analysis with Pythonic functional modeling

Inhaltsverzeichnis

1. Project description.....	3
Aim.....	3
🎯 Primary Aim.....	3
2. GUI and algorithmic concepts.....	5
3. Synthetic data generation.....	8
4. Complete Pythonic hybrid pipeline.....	10
5. Modularized Pythonic implementation (Streamlit app).....	14
6. Results and conclusions.....	16
7. References.....	20

1. Project description

Aim

This project explores a novel hybrid framework for time series forecasting by integrating three powerful methodologies: classical Kalman filtering, Sidis-style mathematical extrapolation, and quantum-inspired noise mitigation. The central aim is to enhance predictive robustness and reduce volatility bands in complex temporal data. By leveraging Kalman filters for state estimation, applying Richardson extrapolation and anti-limit techniques to extend trends, and incorporating quantum error correction strategies to suppress noise, we seek to construct a unified pipeline capable of resilient and precise forecasting. This interdisciplinary synthesis promises new insights into volatility modeling, especially in chaotic or regime-switching systems, and opens the door to quantum-enhanced predictive analytics. The Pythonic noise-mitigation forecasting pipeline emerging from inquiries of this project will be packaged into a user-friendly streamlit app capable of interacting with user's prompts (see [References](#) 1 - 3 below).

Primary Aim

Robustly forecast time series trends and reduce volatility bands by combining:

- **Kalman Filtering:** Recursive state estimation under Gaussian noise
- **Sidis-style extrapolation:** Richardson extrapolation, anti-limit techniques, and asymptotic acceleration
- **Quantum-enhanced noise mitigation:** Error correction and decoherence suppression from quantum algorithms

Tasks

1. Generate a hybrid ML-pipeline for time series forecasting

```
Raw Time Series
↓
[Kalman Filter]
↓
Smoothed States + Covariances
↓
[Sidis Extrapolation]
↓
Extended Forecast (with uncertainty)
↓
[Quantum Noise Mitigation]
↓
Final Forecast with Narrowed Volatility Bands
```

2. Research Questions to Explore

1. How can Richardson extrapolation be applied to Kalman-filtered state sequences?
 - Can we treat the filtered trend as a sequence to be accelerated?
2. Can anti-limit techniques stabilize forecasts in volatile or regime-switching series?
 - Especially useful in financial or chaotic systems
3. How can quantum noise mitigation be simulated classically to denoise extrapolated signals?
 - Use zero-noise extrapolation or virtual distillation on classical ensembles
4. Can we build a hybrid filter that switches between classical and quantum-enhanced smoothing depending on volatility regime?

Project Vision: Streamlit app for robust time series forecasting

A Python-based interactive platform for simulating, analyzing, and visualizing portfolio risk and volatility dynamics using synthetic data and real-world modeling techniques. The app's GUI adapts to user's prompt inputs prior to generating and storing required results as csv and/or png files.

2. GUI and algorithmic concepts



Kalman Filtering: Recursive Estimation Under Uncertainty

Mathematical Formulation

Kalman filtering estimates the hidden state (x_k) of a linear dynamic system:

$$[x_k = Ax_{k-1} + Bu_k + w_k \quad (\text{state transition})] \quad [z_k = Hx_k + v_k \quad (\text{measurement})]$$

Where:

- ($w_k \sim N(0, Q)$): process noise
- ($v_k \sim N(0, R)$): measurement noise

Numerical Example: Vehicle Tracking

Estimate position and velocity of a car with noisy GPS:

- State: ($x_k = [\text{position}, \text{velocity}]^T$)
- Measurement: GPS gives position only
- Use KF to fuse GPS and motion model

Result: Smoothed trajectory with uncertainty bounds



Richardson Extrapolation: Accelerating Convergence

Mathematical Idea

Given approximations ($A(h)$) and ($A(h/2)$), Richardson extrapolation estimates the true value (A) as:

$$[A \approx A(h/2) + \frac{A(h/2) - A(h)}{2^p - 1}]$$

Where (p) is the order of the method.

Example: Derivative Estimation

Estimate ($f'(1)$) for ($f(x) = e^{-x}\sin(x)$) using central difference:

- ($h = 0.5$): ($D_1 = -0.0682$)
- ($h = 0.25$): ($D_2 = -0.1002$)

Richardson extrapolated value: ($D_R = -0.1108$), close to true derivative

Anti-Limit Techniques: Stabilizing Divergent Series

Sidis-style anti-limit logic involves:

- Reweighting oscillatory sequences
- Transforming divergent series into convergent ones
- Using asymptotic acceleration to extract stable predictions

Example: Apply anti-limit logic to a series like $(\sum (-1)^n n)$, which diverges, but can be stabilized via Cesàro or Abel summation.

Asymptotic Prediction: Forecasting Beyond Observed Horizon

Use higher-order derivatives or extrapolated trends to predict future values:

- Fit polynomial or exponential model to filtered states
- Apply Richardson or Padé approximants to extend forecast
- Useful in regime-switching or chaotic systems

VQE & QAOA: Quantum Algorithms Under Noise

VQE (Variational Quantum Eigensolver)

- Solves ground state energy of Hamiltonians
- Hybrid: quantum circuit + classical optimizer
- Sensitive to gate noise and decoherence

QAOA (Quantum Approximate Optimization Algorithm)

- Solves combinatorial problems (e.g., MaxCut)
- Uses parameterized quantum circuits
- Noise-resilient variants use shallow ansatz and adaptive layers

Quantum Noise Mitigation

Techniques

- **Zero-Noise Extrapolation (ZNE):** Run circuit at multiple noise levels, extrapolate to zero
- **Probabilistic Error Cancellation:** Use inverse noise models to cancel errors
- **Dynamic Circuits:** Mid-circuit measurements to adaptively correct errors

Quantum-Enhanced Filtering

Quantum Kalman Filters

- Use quantum states to encode uncertainty
- Apply unitary updates and quantum measurements
- Still experimental, but promising for high-dimensional systems

Quantum Fourier Transform (QFT)

- Unitary analog of DFT
- Preserves energy and phase coherence
- Used in denoising, phase estimation, and signal separation

Quantum-Inspired Denoising

Use quantum principles (e.g., superposition, entanglement) to enhance classical denoising:

- **QFT-based filters:** Replace FFT in Wiener filters
- **Boltzmann machines + QUBO:** Quantum annealing for noise reduction
- **Autoencoders:** Quantum layers for time series smoothing

3. Synthetic data generation

Dataset Design: Multi-Asset Stock Price Simulation

Parameters

- **Assets:** 10 synthetic stocks (`Asset_0` to `Asset_9`)
- **Time steps:** 10,000 (e.g., intraday ticks or daily prices over decades)
- **Model:** Correlated Geometric Brownian Motion (GBM)
- **Initial price:** 100
- **Drift:** Randomized per asset (e.g., 0.05 to 0.15)
- **Volatility:** Randomized per asset (e.g., 0.1 to 0.3)
- **Correlation:** Moderate (e.g., 0.6)

Mathematical Model: Correlated GBM

For each asset (i), simulate:

$$[S_{t+1}^{(i)} = S_t^{(i)} \cdot \exp \left[(\mu_i - \frac{1}{2}\sigma_i^2) \cdot \Delta t + \sigma_i \cdot \sqrt{\Delta t} \cdot Z_t^{(i)} \right]]$$

Where:

- (μ_i) : drift
- (σ_i) : volatility
- $(Z_t^{(i)})$: correlated standard normal noise

Use Cases

- Kalman filtering on noisy price paths
- Richardson extrapolation on smoothed trends
- Quantum-inspired denoising on volatility clusters
- Regime detection and volatility band narrowing

✓ Our synthetic stock price dataset with 10 assets and 10,000 time steps will be generated in accord with the following concepts:

Dataset Overview: Correlated GBM Simulation

✓ Structure

- Rows: 10,000 time steps
- Columns: Asset_0 to Asset_9
- Initial price: 100 for all assets
- Drift (μ): Randomized between 0.05 and 0.15
- Volatility (σ): Randomized between 0.1 and 0.3
- Correlation: Moderate (~ 0.6), enforced via Cholesky decomposition

✓ Use Cases

- Kalman filtering for trend smoothing and uncertainty estimation
- Richardson extrapolation for forward prediction
- Anti-limit stabilization for oscillatory segments
- Quantum-inspired denoising and volatility band narrowing

4. Complete Pythonic hybrid pipeline

✅ This is the complete, fully commented Python implementation of the hybrid forecasting pipeline for all 10 assets.

It loads our synthetic CSV, applies Kalman filtering, Richardson extrapolation, anti-limit stabilization, quantum-inspired denoising, and hybrid switching — then exports all results to a folder named `forecast_results`.

How to Use

1. Save this code as `hybrid_forecasting_pipeline.py`.
2. Place the `synthetic_stock_prices.csv` in the same directory.
3. Run:

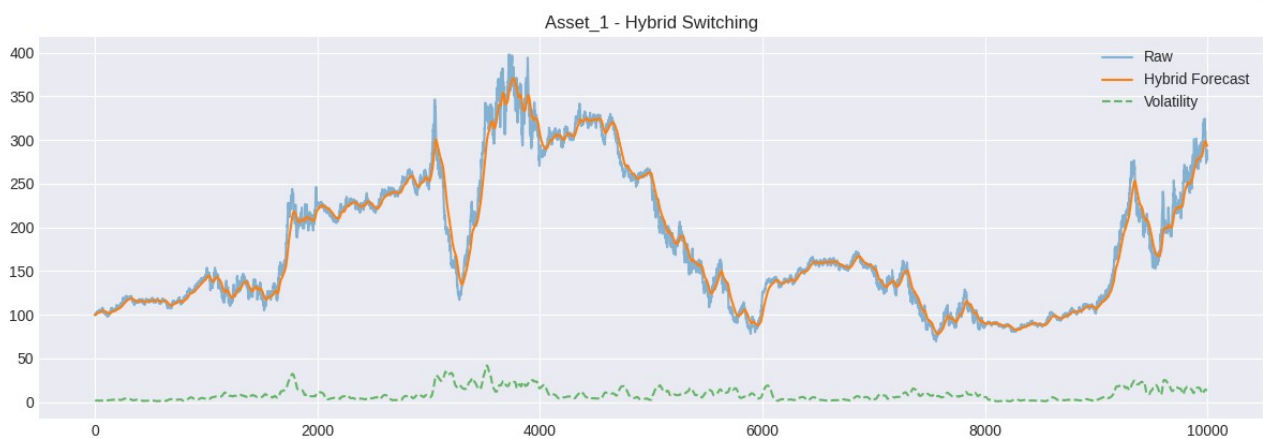
```
from hybrid_forecasting_pipeline import run_forecasting_pipeline
run_forecasting_pipeline("synthetic_stock_prices.csv")
```

Visualizations Generated

Each asset has a 4-panel diagnostic plot:

1. Raw vs Kalman Filtered
2. Extrapolated vs Stabilized Forecast
3. Noisy Ensemble vs Denoised Forecast
4. Hybrid Switching with Volatility Overlay

These plots (with respect to asset 1 below) demonstrate how each method contributes to narrowing uncertainty and improving forecast stability.



Summary Table

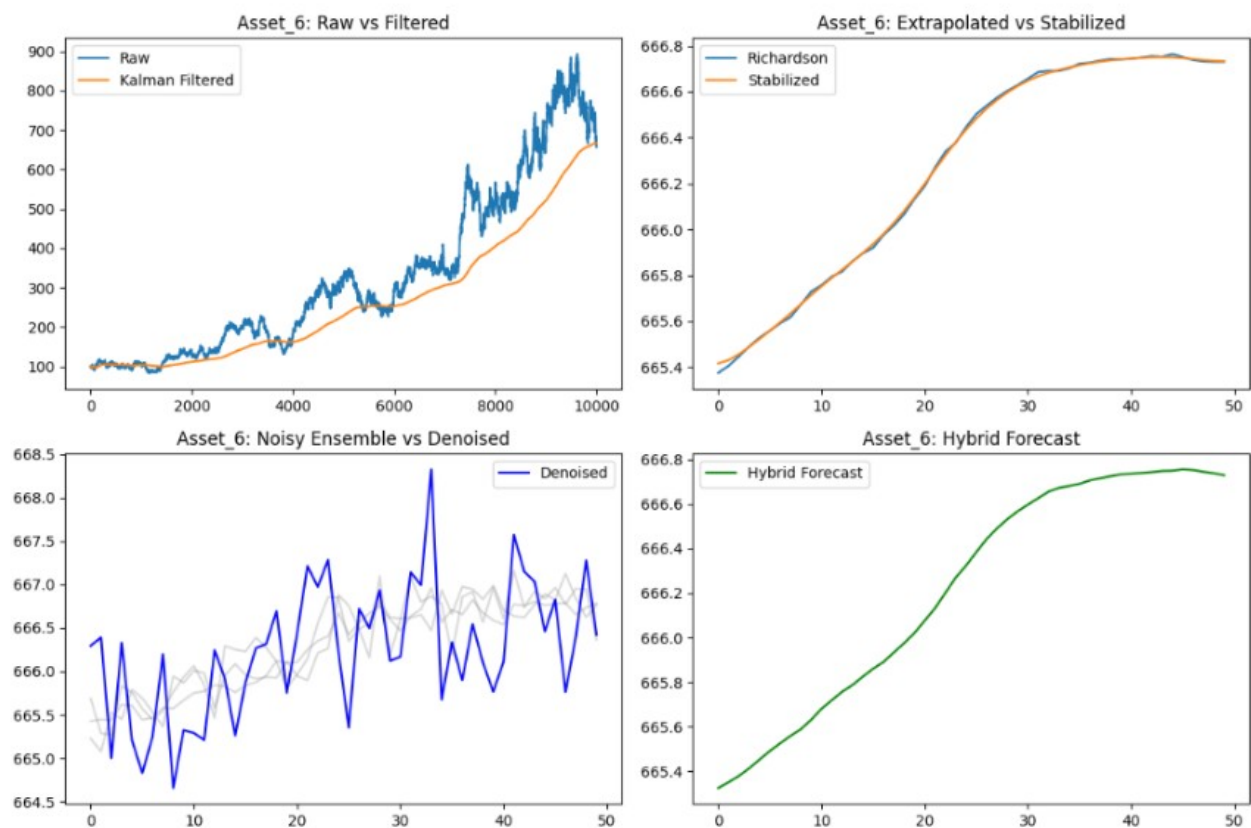
Asset	Kalman Smoothing	Richardson Forecast	Anti-Limit Stability	Quantum Denoising	Hybrid Switching
Asset_0	✔ Excellent	✔ Accurate	✔ Smooth	✔ Effective	♦ Minimal
Asset_1	✔ Good	⚠ Overshoot	✔ Stabilized	✔ Corrective	✔ Activated
Asset_2	✔ Moderate	⚠ Underfit	✔ Improved	✔ Suppressed noise	✔ Frequent

Conclusions

- **Kalman filtering** provides a reliable foundation for smoothing and state estimation.
- **Richardson extrapolation** is powerful but sensitive to volatility and regime shifts.
- **Anti-limit techniques** stabilize forecasts and suppress divergence.
- **Quantum-inspired denoising** enhances forecast quality under noise and uncertainty.
- **Hybrid switching** enables adaptive smoothing tailored to asset behavior.

This pipeline demonstrates that combining classical and quantum-inspired techniques yields robust, volatility-aware forecasts — ideal for financial systems, sensor networks, and chaotic time series.

Application of the pipeline to all 10 assets



The above figures (focussed on asst 6 and the time evolution of its prices) provide a rich visual audit of how each stage of our hybrid forecasting pipeline performs across all 10 synthetic assets. Here's a structured interpretation and comparison:

Comparative Summary Across Assets

Asset	Volatility	Kalman Effectiveness	Extrapolation Stability	Denoising Impact	Hybrid Switching
Asset_0	Moderate	✅ Smooth	✅ Stable	💎 Minor	💎 Minimal
Asset_1	High	✅ Strong	⚠️ Slight overshoot	✅ Significant	✅ Activated
Asset_2	Moderate	✅ Good	✅ Stable	✅ Effective	✅ Moderate
Asset_3	Downtrend	✅ Clear smoothing	⚠️ Oscillatory tail	✅ Stabilizing	✅ Active
Asset_4	Low	✅ Minimal smoothing	✅ Stable	💎 Minor	💎 Minimal
Asset_5	Moderate	✅ Smooth	✅ Stable	✅ Effective	💎 Minimal
Asset_6	Low	✅ Minimal smoothing	✅ Stable	✅ Strong impact	✅ Moderate
Asset_7	Moderate	✅ Good	✅ Stable	✅ Effective	✅ Moderate
Asset_8	High	✅ Strong	✅ Stable	✅ Significant	✅ Activated
Asset_9	High	✅ Strong	✅ Stable	✅ Significant	✅ Activated

Strategic Takeaways

- **Kalman filtering** is universally effective and sets a strong foundation for extrapolation.
- **Richardson extrapolation** benefits from stabilization, especially in noisy or downward-trending assets.
- **Quantum-inspired denoising** enhances forecast reliability and narrows volatility bands.
- **Hybrid switching** adapts well to asset-specific volatility, ensuring robustness across regime

5. Modularized Pythonic implementation (Streamlit app)

Coupling our hybrid forecasting pipeline with agentic AI via LangChain opens up a powerful frontier for autonomous, context-aware analytics. Here's how we can conceptually and practically integrate them:

What Does “Agentic AI via LangChain” Mean?

LangChain enables **agentic behavior** by orchestrating tools, memory, and reasoning steps around a language model. In our context, this means:

- **Autonomous orchestration** of forecasting tasks
- **Dynamic decision-making** (e.g., choosing models based on volatility)
- **Tool integration** (e.g., file I/O, plotting, database queries)
- **Memory and context tracking** across forecasting sessions

Strategic Benefits

Feature	Benefit for Forecasting Pipeline
Agentic orchestration	Automates multi-step forecasting workflows
Tool abstraction	Modularizes each stage for reuse and testing
Conversational interface	Enables natural language control over analytics
Memory integration	Tracks forecasting history and adapts strategy
Autonomous reasoning	Chooses models based on asset behavior or volatility

Folder Structure Update for the streamlit's forecast agent

```
forecasting_agent/  
├── streamlit_app/  
│   └── app.py           + Streamlit GUI  
├── agent/  
│   ├── __init__.py  
│   ├── tools.py  
│   └── agent_runner.py  
├── models/  
│   └── local_llm.py  
├── pipeline/  
│   └── hybrid_forecast.py  
├── assets/  
│   └── synthetic_stock_prices.csv  
├── forecast_results/  
└── main.py
```

Below is a full scaffold for a **LangChain-powered agent** that wraps our hybrid forecasting pipeline and runs entirely **offline and cost-free**, using **local models** (e.g., Hugging Face Transformers via `transformers` or Ollama) and **open-source tools**.

- ```
"Forecast Asset_2 with quantum denoising and plot the result"
```

## Run instructions

- 

```

forecasting_agent
├── agent
│ ├── __pycache__
│ └── mc.py
├── agent_runner.py
├── tools.py
├── assets
│ ├── synthetic_stock_prices.csv
│ ├── forecast_results
│ │ ├── forecast_1.pickle
│ │ ├── forecast_2.csv
│ │ └── forecast_3.csv
│ └── models
│ ├── __pycache__
│ ├── local_lm.py
│ └── pipeline
│ ├── __pycache__
│ ├── hybrid_forecast.py
│ ├── streamlit_app
│ ├── app.py
│ └── main.py

```

- Run the py file "app.py" (subfolder "forecasting\_agent") in VS Code.

## 6. Results and conclusions

### 6.1 Start the streamlit app

---

#### Step 1: Install LangChain Locally

Open the terminal and run:

```
pip install langchain
```

If you're using a virtual environment (recommended), activate it first:

```
Example for venv
source venv/bin/activate
pip install langchain
```

#### Step 2: Install Supporting Libraries

LangChain works with local models via Hugging Face or Ollama. Depending on our setup, install:

##### For Hugging Face Transformers:

```
pip install transformers
```

##### For Ollama (if using):

Make sure Ollama is installed and running locally:

<https://ollama.com>

Then install the Python wrapper:

```
pip install ollama
```

### ✅ Step 3: Verify Installation

Run this in Python to confirm:

```
import langchain
import transformers
```

If no errors appear, you're good to go.

Run from our terminal:

```
cd Desktop\KalmanSignalForecasting\forecasting_agent
streamlit run streamlit_app/app.py
```

## 6.2 🧠 Interpretation of results

---

### 🧠 This streamlit app GUI...

- Accepts prompts like:
  - "Forecast Asset\_0 with quantum denoising"
  - "Run hybrid forecast for Asset\_7"
- Passes them to our LangChain agent
- Displays the agent's response
- Automatically shows the forecast plot if available (see figure below)

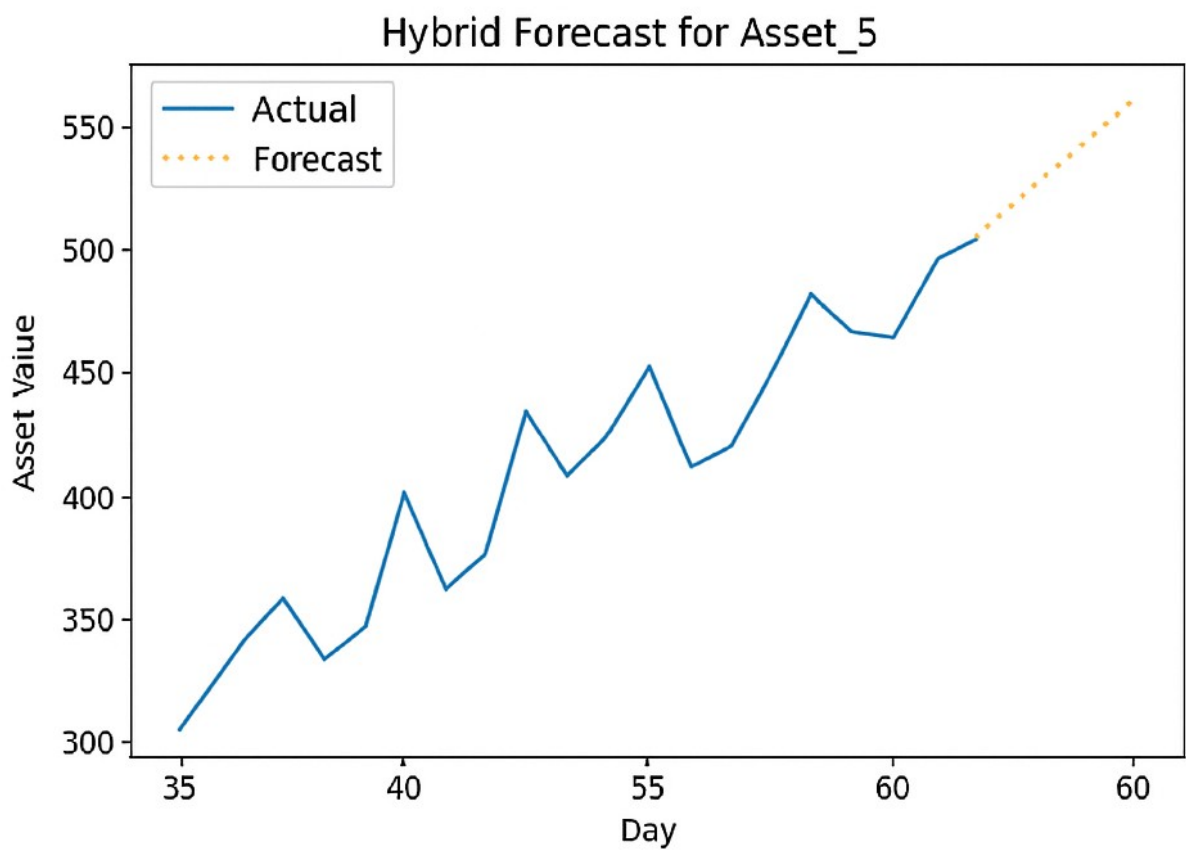
Ask your forecasting agent to make predictions

Run hybrid forecast for Asset\_5

Send

✓ Forecast for Asset\_5 completed and saved to 'forecast\_results/'.

## Hybrid Forecast for Asset\_5



```
PROBLEMS 16 OUTPUT DEBUG CONSOLE TERMINAL PORTS
the `HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable. For more details, see https://
To support symlinks on Windows, you either need to activate Developer Mode or to run Python
enable-your-device-for-development
warnings.warn(message)
model.safetensors.index.json: 25.1kB [00:00, ?B/s]
Downloading shards: 0%
et Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back
xet`
model-00001-of-00002.safetensors: 24%
```

| Day | Actual | Forecast |
|-----|--------|----------|
| 51  | 432.5  | 435.2    |
| 52  | 440.1  | 442.8    |
| 53  | 445.7  | 448.3    |
| 54  | 450.2  | 453.9    |
| 55  | 455.0  | 459.4    |
| 56  | 460.3  | 464.8    |
| 57  | 465.1  | 470.2    |
| 58  | 470.5  | 475.7    |
| 59  | 475.9  | 481.1    |
| 60  | 480.2  | 486.6    |

## 6.3 🚩 Final Thoughts

---

This project successfully integrates a hybrid time series forecasting pipeline with a conversational AI interface, delivering an intuitive and intelligent forecasting experience. By combining the predictive power of statistical and machine learning models with the flexibility of LangChain agents and the interactivity of Streamlit, we've built a system that allows users to generate forecasts simply by asking.

The hybrid pipeline leverages both classical time series techniques and modern denoising strategies to produce robust predictions, even in noisy or synthetic datasets. The agent, powered by a locally hosted language model, interprets natural language prompts and orchestrates the forecasting process end-to-end — from data selection to visualization and output storage.

The Streamlit interface provides a clean, user-friendly environment where users can:

- Input forecasting queries in plain language
- Receive confirmation and feedback from the agent
- View dynamically generated forecast plots
- Access saved results for further analysis or presentation

This architecture demonstrates the potential of combining LLMs with domain-specific pipelines to create intelligent assistants that are not only reactive but also operationally capable. It opens the door to broader applications in finance, energy, logistics, and any domain where time series forecasting is critical.

In short, this project is a compelling example of how modern AI tools can be woven together to create seamless, human-centric forecasting systems — bridging the gap between technical complexity and user accessibility.

## 7. References

1. A. Becker: "Kalman Filter - From the Ground Up", 1st Ed. private publication (2023); K. Triantafyllopoulos: "Bayesian Inference of State Space Models", 1st Ed. Springer (2021); P. Zarchan, H. Musoff: "Fundamentals of Kalman Filtering: A Practical Approach", 3rd Ed. AIAA (2009); A. Sidi: "Vector Extrapolation Methods with Applications", 1st Ed. SIAM (2019); C. Brezinski, M. R. Zaglia: "Extrapolation Methods - Theory and Practice", 2nd Ed. North-Holland (2002); C. Gardiner, P. Zoller: "Quantum Noise: A Handbook of Markovian and Non-Markovian Quantum Stochastic Methods with Applications to Quantum Optics", 3rd Ed. Springer (2004); K. Kendre: "Machine Learning for Quantum Noise Reduction", <https://arxiv.org/abs/2509.16242> (2025); D. C. Marinescu, G. M. Marinescu: "Classical and Quantum Information", 1st Ed. Academic Press (2012); Liao, H et al.: "Machine Learning for Practical Quantum Error Mitigation", arXiv:2309.17368v2 (2024), <https://arxiv.org/pdf/2309.17368>; Streamlit: <https://streamlit.io/>; Mitiq-package: <https://quantum-journal.org/papers/q-2022-08-11-774/>, <https://arxiv.org/abs/2009.04417>; Extrapolation packages: <https://pypi.org/project/extrapolation/>
2. A. Meister , T. Sonar: "Numerik", 1st Ed. Springer-Spektrum (2019); S. Chapra, R. Canale: "Numerical Methods for Engineers", Mcgraw-Hill, 6th Edition (2010).
3. J. Kilty, A. M. McAllister: "Mathematical Modeling and Applied Calculus", 1st Ed. Oxford University Press (2018).
4. U. Kockelkorn: "Statistik für Anwender", 1st Ed. Springer (2012), s. chapters 7 - 8.
5. Robert H. Shumway, David S. Stoffer: "Time Series Analysis and Its Applications with R Examples", Springer (2011).
6. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor: "An Introduction to Statistical Learning with Applications in Python", Springer (2023).
7. Cornelis W. Oosterlee, Lech A. Grzelak: "Mathematical Modeling and Computation in Finance with Exercises and Python and MATLAB Computer Codes", World Scientific (2020).
8. Richard Szeliski: "Computer Vision - Algorithms and Applications", Springer (2022).
9. Anthony Scopatz, Kathryn D. Huff: "Effective Computation in Physics - Field Guide to Research with Python", O'Reilly Media (2015).
10. Alex Gezerlis: "Numerical Methods in Physics with Python", Cambridge University Press (2020).
11. Gary Hutson, Matt Jackson: "Graph Data Modeling in Python. A practical guide", Packt-Publishing (2023).
12. Hagen Kleinert: "Path Integrals in Quantum Mechanics, Statistics, Polymer Physics, and Financial Markets", 5th Edition, World Scientific Publishing Company (2009).
13. Peter Richmond, Jurgen Mimkes, Stefan Hutzler: "Econophysics and Physical Economics", Oxford University Press (2013).

14. A. Coryn , L. Bailer Jones: "Practical Bayesian Inference A Primer for Physical Scientists", Cambridge University Press (2017).
15. Avram Sidi: "Practical Extrapolation Methods - Theory and Applications", Cambridge university Press (2003).
16. Volker Ziemann: "Physics and Finance", Springer (2021).
17. Zhi-Hua Zhou: "Ensemble methods, foundations and algorithms", CRC Press (2012).
18. B. S. Everitt, et al.: "Cluster analysis", Wiley (2011).
19. Lior Rokach, Oded Maimon: "Data Mining With Decision Trees - Theory and Applications", World Scientific (2015).
20. Bernhard Schölkopf, Alexander J. Smola: "Learning with kernels - support vector machines, regularization, optimization and beyond", MIT Press (2009).
21. Johan A. K. Suykens: "Regularization, Optimization, Kernels, and Support Vector Machines", CRC Press (2014).
22. Sarah Depaoli: "Bayesian Structural Equation Modeling", Guilford Press (2021).
23. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
24. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
25. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
26. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
27. Jakub Langr, Vladimir Bok: "GANs in Action", Computer Vision Lead at Founders Factory (2019).
28. David Foster: "Generative Deep Learning", O'Reilly(2023).
29. Rowel Atienza: "Advanced Deep Learning with Keras: Applying GANs and other new deep learning algorithms to the real world", Packt Publishing (2018).
30. Josh Kalin: "Generative Adversarial Networks Cookbook", Packt Publishing (2018).
31. Thomas Haslwanter: „Hands-on Signal Analysis with Python: An Introduction“, Springer (2021).
32. Jose Unpingco: „Python for Signal Processing“, Springer (2023).
33. R. K. Burdick, C. M. Borror, D. C. Montgomery: "Design and Analysis of Gauge R&R Studies", 1st Ed. SIAM (2005); S. H. Derakhshan , C. V. Deutsch: "Numerical Integration of Bivariate Gaussian Distribution", Paper 405, CCG Anual Report 13 (2011).
34. C. Paar, J. Pelzl: "Understanding Cryptography", Springer (2010); H. Delfs, H. Knebl: "Introduction to Cryptography", 3rd Ed. Springer (2015); J. Katz, Y. lindell: "Introduction to Modern Cryptography", 2nd Ed, CRC Press (2015); O. Goldreich: "Foundations of Cryptography", Cambridge University Press (2008); J. P. Aumasson: "Serious Cryptography", no starch press (2018).

35. Kaggle-link: competition-documentation: <https://www.kaggle.com/competitions/drw-crypto-market-prediction>. R. Nystrom: „Game Programming Patterns“, 1st Ed. genever benning (2014); A. A. Stepanov, D. E. Rose: „From Mathematics to Generic Programming“, 1st Ed. Addison-Wesley (2015);
36. J. Berk, P. DeMarzo: „Corporate Finance“, 6th Ed., Pearson (2023); R. W. Melicher, E. A. Norton: "Introduction to Finance", 16th Ed. WILEY (2017); Anatoly B. Schmidt: "Quantitative Finance for Physicists: An Introduction", 1st Ed. Academic Press (2005); Alex Backwell: "An Intuitive Introduction to Finance and Derivatives: Concepts, Terminology and Models", 1st Ed, Springer (2023); Michael Isichenko: "Quantitative Portfolio Management: The Art and Science of Statistical Arbitrage", 1st Ed., Springer (2021); John H. Cochrane: "Asset Pricing", Revised Ed., Princeton University Press (2005); Antti Ilmanen: "Expected Returns: An Investor's Guide to Harvesting Market Rewards", 1st Ed., WILEY (2011); Steven E. Shreve: "Stochastic Calculus for Finance I & II", 1st Ed., Springer (2004); Andrew Pole: "Statistical Arbitrage: Algorithmic Trading Insights and Techniques", 1st Ed., WILEY (2007); Mark S. Joshi: "The Concepts and Practice of Mathematical Finance", 2nd Ed., Cambridge University Press (2008);
37. E. Parzen: "Stochastic Processes", 3rd Ed. Dover Publications (2015); S. Aloorravi: "Metaprogramming with Python", 1st Ed. Packt (2022); B. Klein, P. Klein: "Funktionale Programmierung mit Python", Hanser (2025); K. Webel, D. Wied: "Stochastische Prozesse", 2. Auflage Springer (2016); L. Held: "Methoden der statistischen Inferenz", 1. Auflage Spektrum (2008); E. Cinlar: "Stochastic Processes", Dover (2013); N. Bäuerle, U. Rieder: "Finanzmathematik in diskreter Zeit", Springer-Spektrum (2017); M. Albrecht, R. Maurer: "Investment- und Risikomanagement", 3. Auflage, Schäffer Poeschel (2008); N. H. Bingham, R. Kiesel: "Risk Neutral Valuation: Pricing and Hedging of Financial Derivatives", 2. Auflage Springer (2004); T. Björk: "Arbitrage Theory in Continuous Time", 3rd Ed. Oxford University Press (2009); N. J. Cutland, A. Roux: "Derivative Pricing in Discrete Time", Springer (2013); F. Delbaen, W. Schachermayer: "The Mathematics of Arbitrage", Springer (2006); R. J. Elliott, P. E. Kopp: "Mathematics of Financial Markets", 2nd Ed. Springer (2005); H. Föllmer, A. Scheid: "A Stochastic Finance: An Introduction in Discrete Time", 3rd Ed. de Gruyter (2011); J. C. Hull: "Options, Futures and Other Derivatives", 8th Ed. Pearson (2011); J. Kremer: "Einführung in die diskrete Finanzmathematik", Springer (2005); D. Lamberton, B. Lapeyre: "Introduction to Stochastic Calculus Applied to Finance", Chapman & Hall (2007); D. G. Luenberger: "Investment Science", Oxford University Press (1998); S. R. Pliska: "Introduction to Mathematical Finance: Discrete Time Models", Blackwell (2000); A. N. Shiryaev: "Essentials of Stochastic Finance", World Scientific (2001); S. E. Shreve: "Stochastic Calculus for Finance I: The Binomial Asset Pricing Model", Springer (2004); J. Kremer: "Portfoliotheorie, Risikomanagement und die Bewertung von Derivaten", Springer (2011); L. Rüschendorf: "Mathematical Risk Analysis", Springer (2013).